

CHOOSING AMONG LISTS, SETS, DICTIONARIES, & TUPLES

PROBLEM SOLVING

The classical loop for problem solving with programming:

1. Understand the problem
2. Make a plan (*"formalize the interface"*)
3. Write tests
4. Implement a solution

Hey, look! We can do all of these things now.

TACKLING DUPLICATE PIZZA TOPPINGS

I'm building an app that allows people to order pizzas by dictating their order. I'm able to use software to extract the toppings that people choose into a list.

Sometimes people accidentally repeat toppings they want, which is hard to deal with later. I need a way of creating a "standardized" version of the order that is just a list of the unique toppings.

Step 1: Do you understand what is being asked?

STEP 2: MAKING A PLAN

L11 Can you write the signature for a function that would help you solve this problem? What are the inputs, outputs? Bonus points for type annotations. Try to make sure the signature you write adheres as close as possible to the instructions written here.

I'm building an app that allows people to order pizzas by dictating their order. I'm able to use software to extract the toppings that people choose into a collection.

Sometimes people accidentally repeat toppings they want, which is hard to deal with later. I need a way of creating a "standardized" version of the order that is just a list of the unique toppings.

STEP 2: ~~MAKING A PLAN~~ FORMALIZING INTERFACE

I'm building an app that allows people to order pizzas by dictating their order. I'm able to use software to extract the toppings that people choose into a collection.

Sometimes people accidentally repeat toppings they want, which is hard to deal with later. I need a way of creating a "standardized" version of the order that is just a list of the unique toppings.

```
def standardize_pizza(toppings : list[str]) -> list[str]:  
    ...
```

STEP 3: WRITING TESTS

```
def standardize_pizza(toppings : list[str]) -> list[str]:
```

C12 Practice writing a single unit test for this function. Remember that you'll need:

- test name starting with `test` taking `self` as the only argument
- expected result
- actual result
- assertion statement

Extremely salient exam practice: without referencing notes, could you actually write out on paper a unit test? Find out **now**, not later!!!!

WRITING TESTS

Would we expect first test pass on a correct solution? Would the second one? Ponder.

```
def test_leaves_standard_list_unchanged(self):  
    inputs = ["pineapple", "ham"]  
    expected = ["pineapple", "ham"]  
    actual = standardize_pizza(inputs)  
    self.assertEqual(expected, actual)
```

```
def test_removes_duplicates_short_list(self):  
    inputs = ["pineapple", "ham", "pineapple"]  
    expected = ["pineapple", "ham"]  
    actual = standardize_pizza(inputs)  
    self.assertEqual(expected, actual)
```

WRITING TESTS

Should both tests pass, or only one of them?

```
def test_removes_duplicates_short_list_one(self):  
    inputs = ["pineapple", "ham", "pineapple"]  
    expected = ["pineapple", "ham"]  
    actual = standardize_pizza(inputs)  
    self.assertEqual(expected, actual)
```

```
def test_removes_duplicates_short_list_two(self):  
    inputs = ["pineapple", "ham", "pineapple"]  
    expected = ["ham", "pineapple"]  
    actual = standardize_pizza(inputs)  
    self.assertEqual(expected, actual)
```


Sometimes people accidentally repeat toppings they want, which is hard to deal with later. I need a way of creating a "standardized" version of the order that is just a list of the unique toppings.

- ➡ the instructions are a little underspecified/misleading
 - does the order of toppings actually matter?
 - if so, we should know what order is important. if not, it's hard to specify what the exact correct behavior is.
 - (this is why HW instructions are so extensive... 📖📖📖📖📖)

GOING BACK TO STEP 1

I'm building an app that allows people to order pizzas by dictating their order. I'm able to use software to extract the toppings that people choose into a list.

*Sometimes people accidentally repeat toppings they want, which is hard to deal with later. I need a way of creating a "standardized" version of the order that is just a list of the unique toppings **in alphabetical order**.*

AND THEN BACK TO STEP 3

This is the one we'll keep...



```
def test_removes_duplicates_short_list_one(self):  
    inputs = ["pineapple", "ham", "pineapple"]  
    expected = ["ham", "pineapple"]  
    actual = standardize_pizza(inputs)  
    self.assertEqual(expected, actual)
```

(FINALLY) STEP 4: IMPLEMENT THE SOLUTION

Using what you know about what lists can do, describe the **algorithm** you would use to implement this function. (I'll translate it into code for you—just describe the plan in english.)

FEATURE	DESCRIPTION
Indexing	Access element at index <code>i</code> (0-based, negative indices count from end)
Slicing	Extract sublist from index <code>start</code> to <code>end-1</code>
Length	Returns the number of elements in the list
<code>.index()</code>	Returns the index of the first occurrence of some <code>value</code>
<code>.remove()</code>	Removes the first occurrence of some <code>value</code> from the list

WOW, GREAT JOB!

```
def standardize_pizza(toppings : list[str]) -> list[str]:
    standardized = []
    for topping in toppings:
        if standardized.count(topping) == 0:
            standardized.append(topping)
    return sorted(standardized)
```

```
def standardize_pizza(toppings : list[str]) -> list[str]:
    standardized = []
    for topping in toppings:
        if topping not in standardized:
            standardized.append(topping)
    return sorted(standardized)
```

```
def standardize_pizza(toppings : list[str]) -> list[str]:  
    standardized = []  
    for topping in toppings:  
        duplicate = False  
        for other_topping in standardized:  
            if other_topping == topping:  
                duplicate = True  
        if not duplicate:  
            standardized.append(topping)  
    return sorted(standardized)
```

All of these solutions work, but they have different:

- **simplicity**, related to how many lines of code/levels of indentation are required to write them
- **efficiency**, related to how much time they take to run

Trying the code above on a single input one million times:

```
codio@frienddiet-localindia:~/workspace$ python time_solutions.py  
Solution 1: 0.2352 seconds  
Solution 2: 0.1534 seconds  
Solution 3: 0.3470 seconds
```

The `not in` solution ends up being the fastest by a pretty solid percentage.

- In absolute terms, they're all fast enough for this course—but imagine this magic pizza app really takes off!
 - 🕒 is 💰, and many users mean many calls to all of your functions!
- key takeaway should be writing code that works, but paying attention to the fact that, in Python, *ideas that feel hard to express are often overcomplicated.*

GOING FASTER AND SIMPLIFYING

Who cares about the order of toppings, anyways?

I'm building an app that allows people to order pizzas by dictating their order. I'm able to use software to extract the toppings that people choose into a list.

*Sometimes people accidentally repeat toppings they want, which is hard to deal with later. I need a way of creating a "standardized" version of the order that is just a list of the unique toppings **in any order**.*

Order doesn't matter? **THINK SETS!**

L13 Write an implementation in a single line.

```
def standardize_pizza(toppings : list[str]) -> list[str]:  
    return _____
```

```
def standardize_pizza(toppings : list[str]) -> list[str]:  
    return list(set(toppings))
```

```
codio@frienddiet-localindia:~/workspace$ python time_solutions.py  
Solution 1: 0.2647 seconds  
Solution 2: 0.1893 seconds  
Solution 3: 0.3800 seconds  
Solution 4: 0.1373 seconds (the set solution)
```

Using a set allows us to solve the problem:

- faster than any other idea we had with just lists
- in a single line of code!

NICE!

ACTIVITY

Hackers released a list of 10,000 passwords for PennKeys.

```
princess  
abc123  
666666  
michael  
abc123  
adobe adobe  
azerty  
chocolate  
753951  
macromedia  
121212  
fdsa
```

You and your friends want to count how many common passwords are compromised. Your friends wrote a solution, but you want to make it run more efficiently. Modify the solution so that it is still correct (passing the unit tests) but so that it also runs at least 20% faster! There are a lot of ways you could do this, but keep in mind:

- whether any computation being done here is unnecessary for the logic of the problem to be correct
- whether any choices of data structures could be substituted with others to speed things up.

```
def count_compromised(candidate_passwords: list[str],  
                      hacked_filename: str) -> int:
```



EXAM PRACTICE QUESTIONS

I have a file called `marathon_timings.txt` that maps runner names to their marathon times. e.g.

```
Sofia,270  
Bret,281  
...
```

I want to sort the runners by time and then save the names in order so that I can quickly look up who was in first place, third place, nineteenth place, etc.

I should use a ... to store my runners

- A. list
- B. set
- C. dict
- D. tuple

I have a file called `marathon_timings.txt` that maps runner names to their marathon times. e.g.

```
Sofia,270  
Bret,281  
...
```

I want to be able to look up the marathon times of a runner by their name.

I should use a ... to store my runners & their times.

- A. list
- B. set
- C. dict
- D. tuple

I have a file called `marathon_timings.txt` that maps runner names to their marathon times. e.g.

```
Sofia,270  
Bret,281  
...
```

I want to be able to look up the marathon times of a runner by their name.

can you implement?

```
def build_time_lookup(filename: str) -> dict[str, int]:  
    ...
```



```
def build_time_lookup(filename: str) -> dict[str, int]:
    time_lookup_dict = dict()
    time_file = open(filename, 'r')
    for line in time_file.readlines():
        parsed = line.strip().split(",")
        name = parsed[0]
        time = int(parsed[1])
        time_lookup_dict[name] = time
    return time_lookup_dict
```

fyi, here's a really "terse" way of doing exactly the same:

```
def build_time_lookup(filename: str) -> dict[str, int]:
    time_lookup_dict = dict()
    time_file = open(filename, 'r')
    for line in time_file:
        name, time = line.strip().split(",")
        time_lookup_dict[name] = time
    return time_lookup_dict
```

and an even terser way...

```
def build_time_lookup(filename: str) -> dict[str, int]:
    time_file = open(filename, 'r')
    lines = [l.strip().split(",") for l in time_file.readlines()]
    return {l[0] : int(l[1]) for l in lines}
```

I have a file called `marathon_timings.txt` that maps runner names to their marathon times.

I want to be able to check a name against the names I have stored in this file.

I should use a ... to store my runner names.

- A. list
- B. set
- C. dict
- D. tuple

I have a file called `marathon_timings.txt` that maps runner names to their marathon times.

I want to be able to look up the marathon times of a runner by their names. I also want to be able to add runners & times after I process this file.

I should use a ... to store & update my runner names and times.

- A. list
- B. set
- C. dict
- D. tuple

I have a file called `marathon_timings.txt` that maps runner names to their marathon times.

I want to be able to write a function that returns the name & time of the fastest runner in a single value. It makes the most sense for me to return a ...

- A. list
- B. set
- C. dict
- D. tuple

ANSWERS

1. list
2. dict
3. set
4. dict
5. tuple